# EXPERIMENT NO. 8

**AIM:** To implement banking-style transactions using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT. To simulate partial failures and demonstrate isolation levels and concurrency anomalies using two sessions.

**THEORY:**

Transactions are a group of SQL operations executed as a single logical unit. COMMIT saves changes permanently, ROLLBACK cancels uncommitted changes, and SAVEPOINT allows partial rollback. Isolation levels control how concurrent transactions interact.

**SQL QUERIES / PROGRAM:**

Step 1: Create Database and Table

```
CREATE DATABASE BankingDB;

USE BankingDB;

CREATE TABLE Accounts (

    AccountID INT PRIMARY KEY,

    AccountHolder VARCHAR(50),

    Balance DECIMAL(10,2) NOT NULL

);

INSERT INTO Accounts VALUES

(101, 'Amit', 10000.00),

(102, 'Ravi', 15000.00),

(103, 'Neha', 20000.00);

SELECT * FROM Accounts;
```

Step 2: Demonstrate BEGIN and COMMIT

```
START TRANSACTION;

UPDATE Accounts

SET Balance = Balance - 2000

WHERE AccountID = 101;

UPDATE Accounts
```

```
SET Balance = Balance + 2000
WHERE AccountID = 102;
COMMIT;
SELECT * FROM Accounts;
```

### Step 3: Demonstrate ROLLBACK

```
START TRANSACTION;
UPDATE Accounts
SET Balance = Balance - 5000
WHERE AccountID = 101;
ROLLBACK;
SELECT * FROM Accounts;
```

### Step 4: Demonstrate SAVEPOINT

```
START TRANSACTION;
UPDATE Accounts
SET Balance = Balance - 1000
WHERE AccountID = 101;
SAVEPOINT sp1;
UPDATE Accounts
SET Balance = Balance - 2000
WHERE AccountID = 102;
ROLLBACK TO sp1;
COMMIT;
SELECT * FROM Accounts;
```

### Step 5: Simulate Partial Failure

```
START TRANSACTION;
UPDATE Accounts
SET Balance = Balance - 1000
```

```
WHERE AccountID = 101;

SAVEPOINT transfer_point;

UPDATE Accounts

SET Balance = Balance + 1000

WHERE AccountID = 102;

ROLLBACK TO transfer_point;

COMMIT;

SELECT * FROM Accounts;
```

Step 6: Demonstrate Isolation Levels

```
SET SESSION TRANSACTION ISOLATION LEVEL

READ UNCOMMITTED;

START TRANSACTION;

SELECT * FROM Accounts;

COMMIT;

SET SESSION TRANSACTION ISOLATION LEVEL

READ COMMITTED;

START TRANSACTION;

SELECT * FROM Accounts;

COMMIT;

SET SESSION TRANSACTION ISOLATION LEVEL

REPEATABLE READ;

START TRANSACTION;

SELECT * FROM Accounts;

COMMIT;
```

Step 7: Demonstrate Concurrency Using Two Sessions

Session 1:

```
START TRANSACTION;

UPDATE Accounts
```

```
SET Balance = Balance - 1000

WHERE AccountID = 101; SELECT

* FROM Accounts

WHERE AccountID = 101;

COMMIT;
```

Session 2 (execute before Session 1 commits):

```
START TRANSACTION;

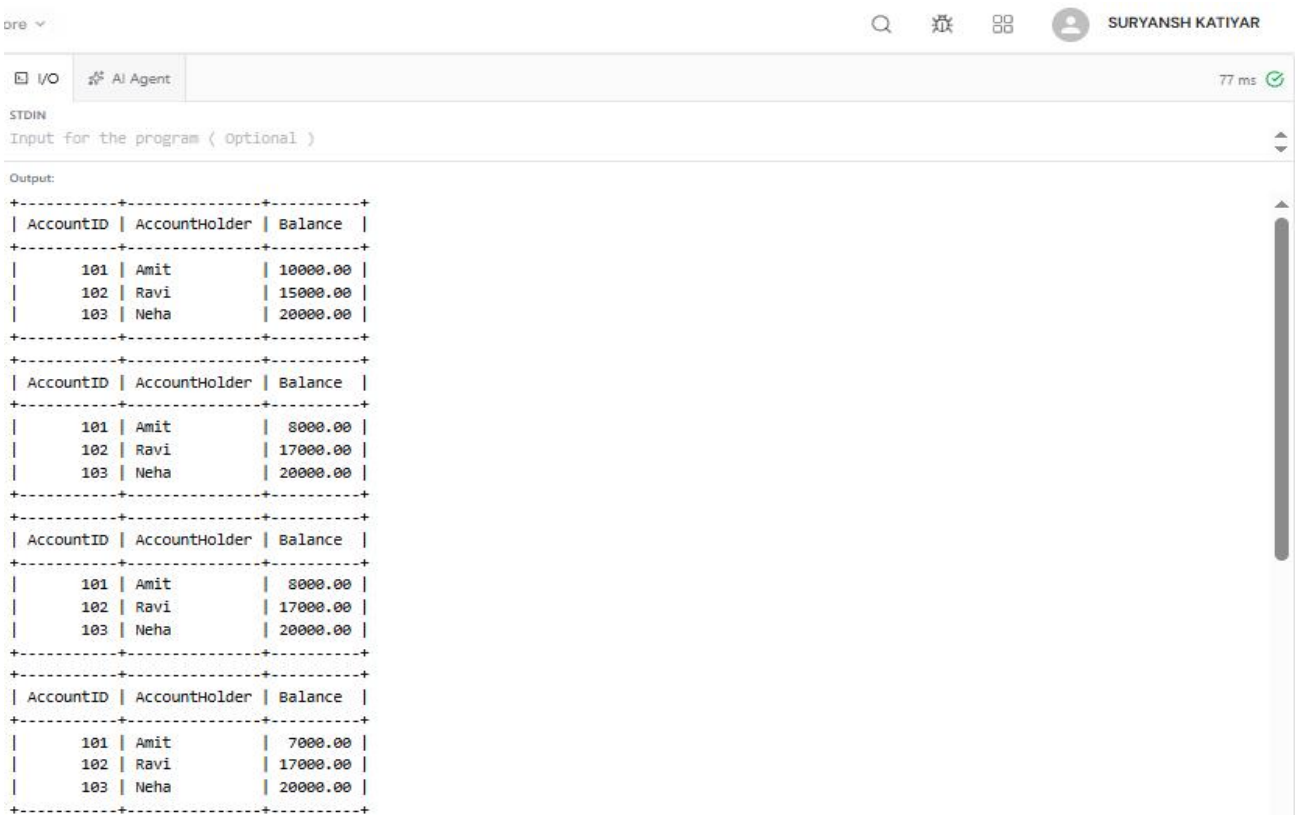
SELECT * FROM Accounts

WHERE AccountID = 101;

COMMIT;
```

The result depends on the isolation level. Under READ UNCOMMITTED, Session 2 may see uncommitted changes. Under READ COMMITTED, it sees only committed changes. Under REPEATABLE READ, repeated reads within a transaction generally return the same committed row version in MySQL InnoDB.

**OUTPUT:**



| AccountID | AccountHolder | Balance |
|---|---|---|
| 101 | Amit | 6000.00 |
| 102 | Ravi | 17000.00 |
| 103 | Neha | 20000.00 |


| AccountID | AccountHolder | Balance |
|---|---|---|
| 101 | Amit | 6000.00 |
| 102 | Ravi | 17000.00 |
| 103 | Neha | 20000.00 |

| AccountID | AccountHolder | Balance |
|---|---|---|
| 101 | Amit | 6000.00 |
| 102 | Ravi | 17000.00 |
| 103 | Neha | 20000.00 |

| AccountID | AccountHolder | Balance |
|---|---|---|
| 101 | Amit | 6000.00 |
| 102 | Ravi | 17000.00 |
| 103 | Neha | 20000.00 |


**RESULT:**

Thus, banking-style transactions were implemented using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT. Partial failures, transaction isolation levels, and concurrency behavior were demonstrated using SQL queries.